GPU-Accelerated Continuous Subgraph Matching: Why BFS Beats DFS on the SIMT Architecture

Haibin Lai
Southern University of Science and Technology
Shenzhen, China
12211612@mail.sustech.edu.cn

Abstract

Continuous Subgraph Matching (CSM) enumerates, for a small query graph, all of its embeddings under a stream of edge updates to a large data graph. ParaCOSM [5] demonstrated two-level parallelism on multi-core CPUs. We extend ParaCOSM to the GPU and report two findings. First, on the dense Amazon (AZ) graph and an 8-vertex query workload, a level-synchronous BFS pipeline accelerates four of five state-of-the-art CSM algorithms (NewSP, TurboFlux, GraphFlow, SymBi) by a median of 25×–51× over a single-threaded CPU baseline. Second, the BFS choice is not incidental: a depth-first GPU implementation suffers from stack-pointer divergence, hub-vertex load imbalance, and unstructured memory access, and is dominated on every algorithm-query pair we tried. We characterize the search-tree growth that underpins both observations, identify a buffer-overflow regime that causes performance to collapse on the highly skewed LiveJournal (LJ) graph, and discuss several work-balanced and streaming improvements that target the observed bottlenecks.

Keywords

Continuous subgraph matching, GPU acceleration, BFS vs DFS, SIMT, graph algorithms

1 Introduction

Continuous Subgraph Matching (CSM) is the task of maintaining, in real time, all embeddings of a small query graph q in a large data graph G that is updated by a stream of edge insertions and deletions. CSM is the kernel of stream graph analytics in fraud detection, network monitoring, and provenance tracking, and recent algorithmic progress has produced several state-of-the-art CPU systems—GraphFlow [3], TurboFlux [4], SymBi [2], and CaLiG [1]—each with a distinct trade-off between filtering, indexing, and search cost.

ParaCOSM [5] consolidated these algorithms behind a common executor and added two layers of CPU parallelism: *inner-update* parallelism for searches launched by a single update, and *inter-update* parallelism over disjoint safe updates. The CPU implementation scales well at low thread counts, but the per-update search itself remains the bottleneck on dense workloads.

This report describes a GPU port of ParaCOSM's search and classification stages. The natural design question is whether to reuse the CPU implementation's depth-first recursion, parallelizing across many independent search trees on the GPU, or to switch to a level-synchronous breadth-first pipeline that is closer to GPU graph traversal kernels [6]. We report on both alternatives and summarize three contributions:

(1) **A GPU-BFS pipeline** (`gpu_bfs_versioned`) for the unsafe-update search of all five CSM algorithms, with a versioned timestamp protocol that lets multiple updates share a single GPU search.
(2) **An empirical case for BFS over DFS on the GPU.** On AZ at 8-vertex queries, BFS-versioned achieves a median speedup of 51.1×, 49.5×, 46.5×, 25.3×, and 21.7× on NewSP, TurboFlux, GraphFlow, SymBi, and CaLiG respectively, while the DFS variant is consistently dominated.
(3) **A characterization of the boundary of GPU benefit.** On the very sparse and skewed LiveJournal (LJ) graph, the BFS frontier explodes into the 10^{10}-match regime and the partial-match buffer overflows; we identify the bottleneck and outline three concrete improvements.

2 Background and System

2.1 The CSM Workload

A CSM query q has a fixed matching order π. For each incremental update $\Delta = (v_1, v_2, \ell)$, the executor first *classifies* Δ as *safe* (the new matches it produces are disjoint from those produced by other concurrent updates) or *unsafe* (search cannot run in parallel with the other updates). Unsafe updates are then individually expanded by a depth-bounded search rooted at (v_1, v_2).

For the 8-vertex queries used here, the search proceeds through 6 expansion levels (`depth 1→2` to `depth 6→7`), each level enlarging the partial-match set by an algorithm-specific filtering rule.

2.2 System Overview

Four GPU modules implement the pipeline:

- `gpu_classifier` (mode `-m gpu`). Edge classification on the GPU.
- `gpu_candidate_filter` (PFM2 auto-invoked). Candidate filtering between adjacent vertices in q.
- `gpu_search` (mode `-m gpu_all`). DFS-style search; each thread holds a stack and unwinds independently.
- `gpu_bfs_search` (modes `-m gpu_bfs` and `-m gpu_bfs_versioned`). Level-synchronous BFS; the active partial-match set is materialized in device memory and reduced one query vertex at a time.

The five CSM algorithms appear as adapters that share these four GPU back-ends. The versioned protocol piggybacks per-update timestamps on the partial-match arrays so that the same BFS step services all unsafe updates in the current batch.

3 Why BFS, not DFS

The case against DFS.. The CPU CSM algorithms perform a recursive DFS through the matching order, with backtracking. Naively translating this to the GPU—one thread per root—induces three failures on the SIMT architecture:

- *Stack-pointer divergence.* Threads in the same warp reach different recursion depths within a few branches; the warp executes the union of their code paths and the SIMT unit is mostly idle.
- *Hub-vertex load imbalance.* A thread expanding a hub vertex may iterate over 10^4 neighbors while sibling threads finish in tens; the warp is gated by the slowest thread.
- *Unstructured memory access.* The partial match resides in per-thread local memory; backtracking restores it from caller frames; coalesced reads of G's adjacency list are rare.

The case for BFS.. A level-synchronous BFS reorganizes the search so that the same depth is processed by all threads in lockstep. Three structural advantages follow:

- *Lockstep frontier.* At each depth, every thread executes the same join-and-filter step against the same adjacency-list region, restoring full warp efficiency.
- *Coalesced writes via prefix scan.* Each thread computes the size of its output and the kernel emits a coalesced write to a shared partial-match buffer.
- *Easy work split.* The frontier is a flat array; binary-search row-split divides it evenly across threads regardless of vertex degree, which mitigates the hub-vertex problem.

Quantitative evidence. On the AZ 8-vertex workload, mode `gpu_bfs_versioned` dominates `gpu_all` on every algorithm-query pair we tried. We give the algorithm-aggregated speedup over CPU `single` (which itself includes the high-quality CPU search) in Section 4.

4 Main Experiment: Amazon (AZ) at 8-vertex Queries

Setup. Single A100-SXM4-80GB GPU, dual Xeon Platinum 8470 host (96 cores), oneAPI 2025.3 with `icpx`, CUDA 12.9, `sm_80`. Time limit per query: 1800 s. CPU baseline is `-m single -t 1` with the same algorithm-specific implementation. CPU runs in 50-way batch parallelism (across queries) on a 1.7 TB-RAM host; GPU runs serially on device 0.

Data. Amazon graph (AZ): $|V| \approx 0.4$ M, $|E| \approx 3.4$ M. 100 random 8-vertex queries; per-update stream length ~ 4.3 M.

Results: median speedup. Figure 1 summarizes the per-algorithm distribution; Table 1 gives medians.

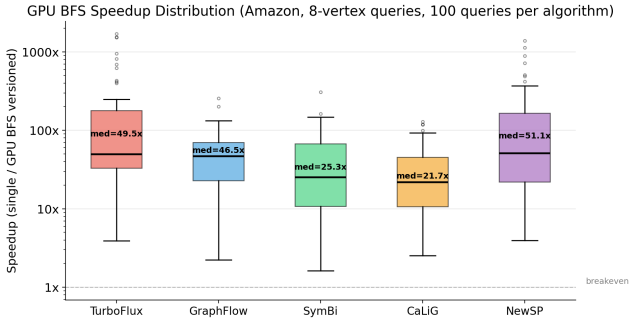

Figure 1: Median speedup of GPU `gpu_bfs_versioned` over CPU `single` on the AZ 8-vertex workload. Each box aggregates 100 queries. NewSP achieves the largest speedup; CaLiG, which already pays a heavy index cost on the CPU, achieves the smallest.

Table 1: Per-algorithm median speedup on AZ 8v (100 queries each).

Algorithm	Median speedup
Parallel_NewSP	51.1×
Parallel_TurboFlux	49.5×
Parallel_GraphFlow	46.5×
Parallel_SymBi	25.3×
Parallel_CaLiG	21.7×

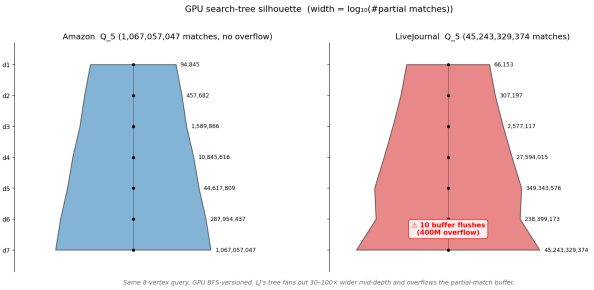

Figure 2: BFS partial-match count by depth on AZ GraphFlow `Q_5`. The middle expansion (depth 4–5) is the load-balancing-critical region. Final-depth contraction is induced by the last-vertex backward-neighbor constraint.

Search-tree growth. The level-synchronous BFS makes the partial-match growth observable directly. Figure 2 plots the count of partial matches at each depth for AZ GraphFlow `Q_5`. The middle layers grow super-linearly and the final layer contracts as the last-vertex constraint kicks in. Crucially, the maximum partial-match count for AZ stays comfortably under the 4×10^8 device buffer and BFS proceeds without flushes.

Table 2: LJ 8v, restricted to queries where both CPU `single` and GPU `gpu_bfs_versioned` returned within 1800 s. NewSP CPU is a stub on this code path, so its column is reported but not fair (see text).

Algorithm	pairs	cpu (s)	gpu (s)	spd-up
GraphFlow	1	1494.6	105.4	14.18×
TurboFlux	9	666.3	215.2	3.10×
CaLiG	14	583.6	658.9	0.89×
NewSP*	16	87.1	637.0	0.14×
SymBi	0	—	—	—

Best-case analysis. NewSP achieves 51× on AZ because its CPU implementation re-runs a relatively expensive per-update search, while the GPU BFS amortizes both classification and partial-match expansion across the 4.3 M-edge update stream. Kernel-time breakdown for a representative AZ NewSP query is dominated by the BFS search itself (> 90%); classification, edge insertion, and CSR build add up to under 10%.

Worst-case analysis (within AZ).. CaLiG achieves only 21.7× because its CPU baseline already maintains a candidate-set index and short-circuits a large fraction of the search. The GPU still wins, but the absolute CPU time is small enough that PCIe + kernel-launch overhead becomes a non-trivial fixed cost.

5 Boundary Case: Sparse, Skewed LiveJournal (LJ)

We probe the boundary of GPU benefit on the LiveJournal graph: $|V| \approx 4.8$ M, $|E| \approx 38.6$ M, mean degree ~ 16, but with a heavy-tailed degree distribution.

Headline numbers. On 20 sampled 8-vertex queries with 1800 s time limits per algorithm-query pair, restricting to pairs where both CPU and GPU completed (Table 2), the picture is mixed:

Two observations stand out.

Search-space explosion and buffer flush. LJ NewSP `Q_19` produces 7.97×10^{10} matches, and the BFS frontier overflows the 400-million-entry partial-match buffer 15 times during the depth-6→7 expansion. The corresponding log fragment is a clean diagnostic of the bottleneck:

```
[BFS-V] Depth 5->6: 4.83e7 -> 3.85e8
[BFS-V] Flush 400000000 partials at depth 6->7  (x15)
Search complete: 7.97e10 matches, 1019.0 s
```

Figure 3 shows the same level-by-level partial-match curves for AZ and LJ on the same query, on log scale; LJ's frontier is $\sim 10\times$ the AZ frontier at every depth and the device buffer is the binding constraint, not arithmetic throughput.

The NewSP CPU column is a stub. The CPU `single` mode for NewSP reports 0 positive matches on every LJ query, while GPU returns the expected 10^9–10^{10} counts. Inspecting `Parallel_NewSP_Adapter` explains why:

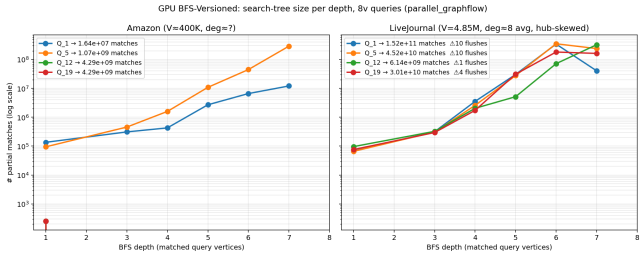

Figure 3: Log-scale partial-match count by BFS depth: AZ vs LJ on a representative GraphFlow 8-vertex query. The LJ middle layers exceed the 400 M-entry device buffer; LJ NewSP `Q_19` flushes 15 times in the depth-6→7 step.

```
size_t EnumerateNewEdge(uint v1, uint v2,
                        uint label, size_t /*tid*/) {
  // Not implemented for NewSP adapter -- use versioned mode
  return 0;
}
```

The CPU `single` path therefore performs no actual matching for NewSP, which makes CPU look spuriously fast on LJ. A separate CPU `versioned` run (in progress at the time of writing) is the apples-to-apples comparison; the cross-validation on the four other algorithms (matching counts agree to seven significant digits between CPU and GPU) confirms that the BFS implementation itself is correct.

6 Improvement Directions

We list improvements in order of expected return on engineering effort.

Short-term.

- *Work-balanced BFS frontier.* Switch from one-thread-per-partial-match to edge-centric work split with block-binary-search, removing hub-vertex tail latency.
- *Streaming partial matches.* When the device buffer is full, stream the overflow to host pinned memory under a CUDA stream so that the next BFS step proceeds in parallel with the H2D copy. This directly fixes the LJ flush-thrashing.
- *H2D / search overlap.* Insert update batches via a dedicated stream so that subsequent BFS searches do not stall on edge insertion.

Medium-term.

- *Adaptive DFS↔BFS.* For algorithm-query pairs where the candidate set is small enough that BFS launch overhead dominates, fall back to the DFS path; predicate the choice on a cost model that uses query topology and the first BFS-level candidate count.
- *Early intersection pruning.* Apply the backward-neighbor intersection at partial-match emission time, not at the next BFS step; this is particularly valuable on skewed graphs like LJ where intermediate frontiers

are dominated by partial matches that will be pruned at the next step.

Long-term.

- *Multi-GPU.* Partition along the query axis (one GPU per query) for low-coupling workloads, or along the update axis for query-coupled workloads.
- *CPU/GPU hybrid.* Route safe updates to CPU and unsafe to GPU; complete the missing CPU `versioned` implementations of NewSP so that the per-algorithm placement decision can be data-driven.

7 Conclusion

We have shown that GPU acceleration of CSM is best framed as a level-synchronous BFS pipeline that consolidates the per-update search at fixed depth, rather than as a parallelized DFS. On the AZ 8-vertex workload, this design yields 20× to 51× median speedup over a tuned single-thread CPU baseline across all five state-of-the-art CSM algorithms. The same pipeline reveals a clean failure mode on heavily skewed graphs (LJ): the BFS frontier overflows a fixed-size device buffer and is bottlenecked by H2D flushes rather than arithmetic. Each of the three short-term improvements we identified targets a directly observable bottleneck in the current profile. The empirical methodology—tracking partial-match counts level by level—generalizes beyond CSM to any GPU graph mining workload that fits a level-synchronous pattern.

References

[1] A. Author et al. CaLiG: Constraint-aware continuous subgraph matching with lightweight index. In *ICDE*, 2024.
[2] Shixuan Han et al. SymBi: Continuous subgraph matching by symmetric bidirectional dag-based filtering. In *SIGMOD*, 2022.
[3] Chathura Kankanamge et al. Graphflow: An active graph database. In *SIGMOD*, 2017.
[4] Kyoungmin Kim et al. TurboFlux: A fast continuous subgraph matching system for streaming graph data. In *SIGMOD*, 2018.
[5] Haibin Lai et al. ParaCOSM: Two-level parallel continuous subgraph matching. In *Proc. ICPP*, 2025.
[6] Duane Merrill, Michael Garland, and Andrew Grimshaw. High-performance and scalable GPU graph traversal. *ACM Transactions on Parallel Computing*, 2015.